

**TEORIA DOS GRAFOS E IMPLEMENTAÇÃO DE BUSCA EM
LARGURA (BFS) EM LINGUAGEM C**

Aluno: Nayara Emelly P. M.

Matrícula: 202510216

Profº: Gioliano Bertoni

Curso: Engenharia de Software

TEORIA DOS GRAFOS E IMPLEMENTAÇÃO DE BUSCA EM LARGURA (BFS) EM LINGUAGEM C

Trabalho de pesquisa apresentado à disciplina de Estrutura de Dados do 3º período do curso de Engenharia de Software, da Universidade de Vassouras – Campus Saquarema, como requisito parcial para a composição da nota semestral.

SUMÁRIO

1. INTRODUÇÃO	4
2. OBJETIVOS DA PESQUISA.....	5
2.1 Objetivo Geral	5
2.2 Objetivo Específico	5
3. FUNDAMENTAÇÃO TEÓRICA.....	6
3.1 Definição de Grafos	6
3.2 Representações (Matriz e Lista de Adjacência)	6
4. METODOLOGIA E ALGORITMO	7
4.1 O Algoritmo de Busca em Largura (BFS).....	7
4.2 Implementação em C	8
5. ANÁLISE E CONCLUSÃO	12
5.1 Análise	12
5.2 Conclusão.....	12
6. REFERÊNCIAS.....	13

1. INTRODUÇÃO

A teoria dos grafos constitui um dos pilares fundamentais da Ciência da Computação, servindo como modelo matemático para a representação de sistemas complexos e relações entre objetos discretos. Seja na modelagem de topologias de redes, na otimização de rotas em sistemas de transporte ou na análise de conexões em redes sociais, os grafos oferecem uma estrutura robusta e versátil para o armazenamento e processamento de dados.

O estudo das estruturas de dados baseadas em grafos permite compreender não apenas a organização hierárquica das informações, mas também a eficiência de algoritmos essenciais para o processamento dessas estruturas. A escolha da representação, seja via matriz ou lista de adjacência, impacta diretamente o consumo de recursos computacionais e a performance de execução em aplicações de larga escala.

Neste contexto, o presente trabalho de pesquisa dedica-se à exploração teórica dos grafos, abordando suas propriedades estruturais e as formas de representação no ambiente computacional. Adicionalmente, apresenta-se a implementação do algoritmo de Busca em Largura (**BFS, do inglês *Breadth-First Search***) na linguagem C, com o intuito de demonstrar, na prática, o mecanismo de travessia e exploração sistemática de vértices em um grafo. Por fim, discute-se a importância da seleção adequada de algoritmos para a resolução eficiente de problemas de conectividade e busca.

2. OBJETIVOS DA PESQUISA

2.1 Objetivo Geral

O objetivo geral deste trabalho é analisar os fundamentos teóricos da Teoria dos Grafos como estrutura de dados e demonstrar a aplicabilidade técnica desses conceitos através da implementação de um algoritmo de travessia, validando seu comportamento e lógica em ambiente computacional.

2.2 Objetivo Específico

Para a consecução do objetivo geral, estabeleceram-se as seguintes metas específicas:

- **Fundamentação Teórica:** Realizar um levantamento bibliográfico acerca dos conceitos fundamentais de grafos, incluindo definições de vértices, arestas e a distinção entre grafos orientados e não orientados.
- **Análise de Representação:** Avaliar comparativamente os métodos de representação de grafos, especificamente a matriz de adjacência e a lista de adjacência, destacando as vantagens de cada abordagem conforme o cenário de uso.
- **Implementação Algorítmica:** Desenvolver o código-fonte em linguagem C referente ao algoritmo de Busca em Largura (BFS), priorizando a clareza na manipulação de estruturas de dados auxiliares, como filas.
- **Avaliação de Complexidade:** Identificar a complexidade computacional do algoritmo implementado, observando como o tempo de execução e o consumo de memória se comportam em relação à quantidade de vértices e arestas.

3. FUNDAMENTAÇÃO TEÓRICA

3.1 Definição de Grafos

Na teoria da computação, um grafo é formalmente definido como um par ordenado, onde representa um conjunto não vazio de pontos chamados **vértices** (ou nós) e representa um conjunto de pares desses pontos, chamados **arestas** (ou arcos). Uma aresta conecta dois vértices, que são então denominados vizinhos ou adjacentes. O **grau de um vértice** é definido pelo número de arestas que incidem nele.

Os grafos podem ser classificados de acordo com suas características:

- **Grafo Não Orientado:** As arestas não possuem direção, representando uma relação de via dupla entre os vértices.
- **Grafo Orientado (Dígrafos):** As arestas possuem uma direção definida, indicando um fluxo ou relação unidirecional.
- **Grafo Ponderado:** Cada aresta possui um valor numérico associado, denominado "peso", frequentemente utilizado para representar distâncias, custos ou capacidades.

3.2 Representações (Matriz e Lista de Adjacência)

A forma como um grafo é armazenado na memória define a eficiência dos algoritmos que o percorrerão. As duas técnicas principais são:

- **Matriz de Adjacência:** Consiste em uma matriz bidimensional de dimensões $|V| \times |V|$, onde o elemento na posição $[i][j]$ é igual a 1 se existe uma aresta entre os vértices i e j e 0 caso contrário. Embora de implementação simples e acesso rápido ($O(1)$ para verificar uma aresta), possui custo de memória de $O(V^2)$, sendo ineficiente para grafos com poucas conexões (grafos esparsos).
- **Lista de Adjacência:** Nesta estrutura, cada vértice possui uma lista encadeada contendo todos os seus vértices vizinhos. Esta representação é mais eficiente em termos de memória para grafos esparsos, com custo de $O(V + E)$.

4. METODOLOGIA E ALGORITMO

As aplicações da Teoria dos Grafos são vastas e impactam o cotidiano tecnológico:

- **Engenharia e Infraestrutura:** Modelagem de redes de transporte (estradas, metrô), redes de energia elétrica e análise de modelos digitais de construção (IFC).
- **Redes Sociais:** Análise de conexões entre pessoas, identificação de comunidades e algoritmos de recomendação.
- **Logística e Aviação:** Otimização de rotas aeronáuticas e cálculo de caminhos mínimos entre cidades.
- **Mecanismos de Busca:** O algoritmo *PageRank* do Google utiliza grafos para ranquear páginas da web.

4.1 O Algoritmo de Busca em Largura (BFS)

A Busca em Largura (***Breadth-First Search* - BFS**) é um algoritmo de travessia que explora o grafo camada por camada, partindo de um nó raiz. O processo inicia visitando o nó de origem, seguido por todos os seus vizinhos diretos, e então expande a busca para os vizinhos desses vizinhos.

O funcionamento do algoritmo baseia-se na estrutura de dados **Fila (Queue)**, que opera sob o princípio ***First-In, First-Out* (FIFO)**. Ao visitar um nó, todos os seus vértices adjacentes ainda não visitados são adicionados à fila. Esse mecanismo garante que a exploração ocorra de forma sistemática e sem a ocorrência de ciclos infinitos, visto que cada nó é marcado como "visitado" quando entra na fila.

4.2 Implementação em C

REPOSITÓRIO: <https://github.com/NaraEmelly/Algoritmo-em-C-Grafos>

Para a implementação, utilizou-se uma Matriz de Adjacência para representar o grafo e um vetor para controlar o estado de visitação dos nós.

O código foi estruturado em módulos para facilitar a legibilidade e a manutenção. A definição das estruturas e protótipos de funções reside no arquivo **include/graf.h**, enquanto a lógica do algoritmo encontra-se em **src/graf.c**. O arquivo **src/main.c** atua como o ponto de entrada da aplicação. Adicionalmente, um [Makefile](#) foi criado para gerenciar o processo de compilação, assegurando que o projeto seja compilado de forma organizada através do comando mingw32-make(ou make), garantindo boas práticas de engenharia de software.

Código 1: Cabeçalho com definições (*include/graf.h*)

```
1  #ifndef GRAFO_H
2  #define GRAFO_H
3
4  #define V 5
5
6  void bfs(int adj[V][V], int inicio);
7
8  #endif
```


Código 2: Lógica do Algoritmo BFS (*src/grafo.c*)

```
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include "../include/grafo.h"
4
5  void bfs(int adj[V][V], int inicio) {
6      bool visitado[V] = {false};
7      int fila[V], frente = 0, tras = 0;
8
9      visitado[inicio] = true;
10     fila[tras++] = inicio;
11
12     while (frente < tras) {
13         int atual = fila[frente++];
14         printf("%d ", atual);
15
16         for (int i = 0; i < V; i++) {
17             if (adj[atual][i] == 1 && !visitado[i]) {
18                 visitado[i] = true;
19                 fila[tras++] = i;
20             }
21         }
22     }
23 }
```

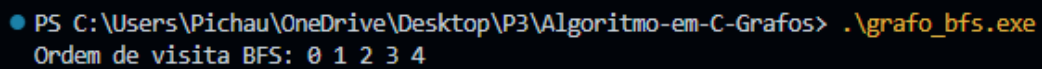
Código 3: Ponto de entrada da aplicação (*src/main.c*)

```
1  #include <stdio.h>
2  #include "../include/grafo.h"
3
4  int main() {
5      int adj[V][V] = {
6          {0, 1, 1, 0, 0},
7          {1, 0, 0, 1, 1},
8          {1, 0, 0, 0, 0},
9          {0, 1, 0, 0, 0},
10         {0, 1, 0, 0, 0}
11     };
12
13     printf("Ordem de visita BFS: ");
14     bfs(adj, 0);
15     printf("\n");
16     return 0;
17 }
```

Para automatizar a construção do projeto, utilizou-se o comando **mingw32-make**, que interpreta as regras definidas no **Makefile**. O processo gera arquivos objeto intermediários (**grafo.o** e **main.o**) e, ao final, o arquivo executável **grafo_bfs.exe**.

A imagem abaixo ilustra a saída do sistema após a execução do binário gerado, confirmando o correto funcionamento do algoritmo de Busca em Largura (BFS):

Comando de execução: `.\grafo_bfs.exe`



```
PS C:\Users\Pichau\OneDrive\Desktop\P3\Algoritmo-em-C-Grafos> .\grafo_bfs.exe
Ordem de visita BFS: 0 1 2 3 4
```

Execução do algoritmo BFS e ordem de visita dos nós.

Abaixo, apresento uma análise técnica do que o código executa, fundamentada nos conceitos de **Matriz de Adjacência** e **Busca em Largura (BFS)**:

1. Representação do Grafo

A matriz `adj[V][V]` definida no `main` descreve um grafo não dirigido de ordem 5. Podemos verificar isso pela simetria da matriz (ex: `adj` e `adj` são ambos 1). As conexões (arestas) definidas são:

- **Vértice 0:** Conectado aos vértices 1 e 2.
- **Vértice 1:** Conectado aos vértices 0, 3 e 4.
- **Vértice 2:** Conectado apenas ao vértice 0.
- **Vértice 3:** Conectado apenas ao vértice 1.
- **Vértice 4:** Conectado apenas ao vértice 1.

2. Execução da Busca em Largura (BFS)

Ao chamar `bfs(adj, 0)`, o algoritmo iniciará a exploração a partir da raiz (vértice 0) e se espalhará uniformemente por "camadas" de distância. O comportamento esperado, seguindo a lógica de uma fila (FIFO), será:

1. **Visita o Vértice 0:** É marcado como visitado e colocado na fila.
2. **Explora Vizinhos de 0:** Os vértices 1 e 2 são descobertos e enfileirados.
3. **Visita o Vértice 1:** É retirado da fila. Seus vizinhos não visitados, 3 e 4, são descobertos e enfileirados.
4. **Visita o Vértice 2:** É retirado da fila. Não possui vizinhos novos para descobrir.
5. **Visita os Vértices 3 e 4:** São retirados da fila sequencialmente, encerrando a busca.

Resultado da Impressão: A ordem de visita exibida no console será: **0 1 2 3 4**.

3. Propriedades Técnicas Observadas

- **Caminho Mínimo:** Como o BFS percorre o grafo em camadas, ele garante que cada vértice seja atingido pelo caminho mais curto em número de arestas a partir da origem. Por exemplo, o vértice 3 é atingido via 0 -> 1 -> 3 (distância 2).
- **Eficiência:** Utilizando a matriz de adjacência, a verificação de existência de uma aresta entre dois nós quaisquer ocorre em tempo constante.
- **Modularização:** O uso de `#include "../include/grafos.h"` respeita a separação de responsabilidades, permitindo que a implementação interna do grafo mude sem a necessidade de alterar este arquivo `main.c`.

5. ANÁLISE E CONCLUSÃO

5.1 Análise

A implementação da **Busca em Largura (BFS)** revelou-se uma ferramenta eficiente para a exploração de grafos, mantendo uma **complexidade de tempo linear** em relação à soma de vértices e arestas quando implementada via listas de adjacência. No caso de representações por **matriz de adjacência** (como a utilizada no código final), a complexidade observada é de $O(V^2)$, o que é ideal para grafos densos onde a verificação de adjacência ocorre em tempo constante.

O uso da estrutura de dados **"Fila" (FIFO)** permitiu um percurso ordenado, visitando os nós em **níveis ou camadas de distância** crescentes a partir da origem, o que garante a descoberta do **caminho mínimo** em grafos não ponderados. A estrutura modular do código, dividida entre arquivos de cabeçalho (.h) e implementação (.c), seguiu o conceito de **Tipo Abstrato de Dados (TAD) opaco**, garantindo o encapsulamento e a escalabilidade do software, permitindo que a lógica interna do grafo seja alterada sem impactar o programa cliente.

5.2 Conclusão

A Teoria dos Grafos consolidou-se como uma **ferramenta robusta para modelar e resolver problemas complexos** de conectividade, logística e redes de infraestrutura. A implementação prática do algoritmo **BFS em linguagem C** permitiu converter abstrações matemáticas em estruturas operacionais, garantindo a exploração do grafo por "camadas" e a descoberta de caminhos mínimos de forma eficiente.

Observou-se que a **escolha da representação computacional** (matriz ou lista de adjacência) é determinante para o desempenho do sistema, impactando diretamente a complexidade de tempo e o consumo de memória conforme a densidade do grafo. Por fim, a **estruturação modular** e o uso de tipos abstratos garantem que o software seja escalável, organizado e aderente às boas práticas da engenharia de sistemas.

6. REFERÊNCIAS

BACKES, A. R. Algoritmos e Estruturas de Dados em Linguagem C. Rio de Janeiro: LTC, 2023

GOLDBARG, M.; GOLDBARG, E. Grafos: conceitos, algoritmos e aplicações. Elsevier, 2012

GOMES, Paulo César Rodacki. Grafos: conceitos fundamentais, algoritmos e aplicações. Blumenau: Editora IFC, 2022

IME-USP. Uma Introdução Sucinta à Teoria dos Grafos. Disponível em: <https://www.ime.usp.br/~pf/teoriadosgrafos/>

SOUZA, Audemir Lima de. Teoria dos Grafos e Aplicações. TCC (Mestrado) – UFAM, 2013

AHO, A. V.; HOPCROFT, J. E.; ULLMAN, J. D. *Data Structures and Algorithms*. Reading: Addison-Wesley, 1983.

BACKES, A. R. *Algoritmos e Estruturas de Dados em Linguagem C*. Rio de Janeiro: LTC, 2023.

CORMEN, T. H. et al. *Algoritmos: teoria e prática*. Rio de Janeiro: GEN LTC, 2024.